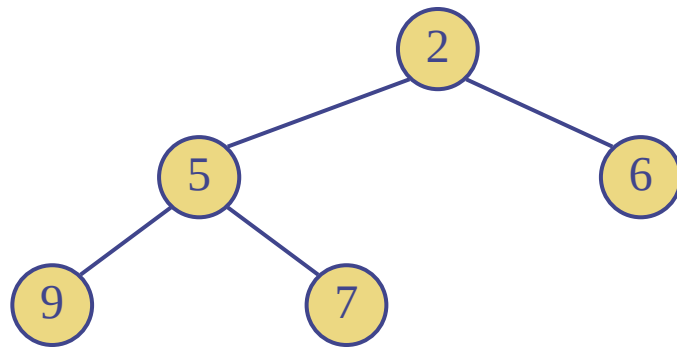


Heaps



Recall Priority Queue ADT

- ❑ A priority queue stores a collection of items
- ❑ Each **item** is a pair (key, value)
- ❑ Main methods of the Priority Queue ADT
 - **add(k, x)**
inserts an item with key k and value x
 - **remove_min()**
removes and returns the item with smallest key
- ❑ Additional methods
 - **min()**
returns, but does not remove, an item with smallest key
 - **len(), is_empty()**
- ❑ Applications:
 - Standby flyers
 - Auctions
 - Stock market

Recall PQ Sorting



- We use a priority queue
 - Insert the elements with a series of **add** operations
 - Remove the elements in sorted order with a series of **remove_min** operations
- The running time depends on the priority queue implementation:
 - Unsorted sequence gives selection-sort: $O(n^2)$ time
 - Sorted sequence gives insertion-sort: $O(n^2)$ time
- Can we do better?

Algorithm **PQ-Sort**(S, C)

Input sequence S , comparator C for the elements of S

Output sequence S sorted in increasing order according to C

$P \leftarrow$ priority queue with comparator C

while $\neg S.is_empty()$

$e \leftarrow S.remove(S.first())$

$P.add(e, e)$

while $\neg P.is_empty()$

$e \leftarrow P.remove_min().key()$

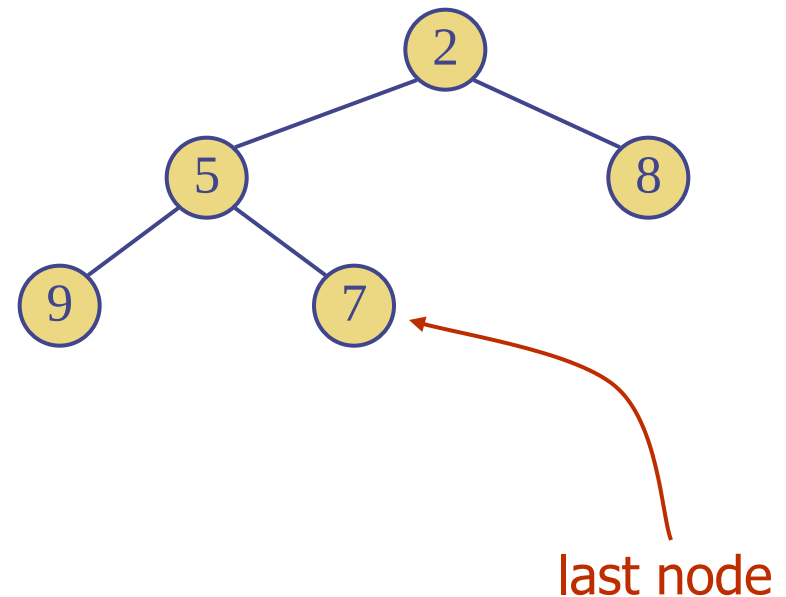
$S.add_last(e)$

Heaps



- A heap is a binary tree storing keys at its nodes and satisfying the following properties:
- **Heap-Order:** for every internal node v other than the root, $key(v) \geq key(parent(v))$
- **Complete Binary Tree:** let h be the height of the heap
 - for $i = 0, \dots, h - 1$, there are 2^i nodes of depth i
 - at depth $h - 1$, the internal nodes are to the left of the external nodes

- The **last node** of a heap is the rightmost node of maximum depth



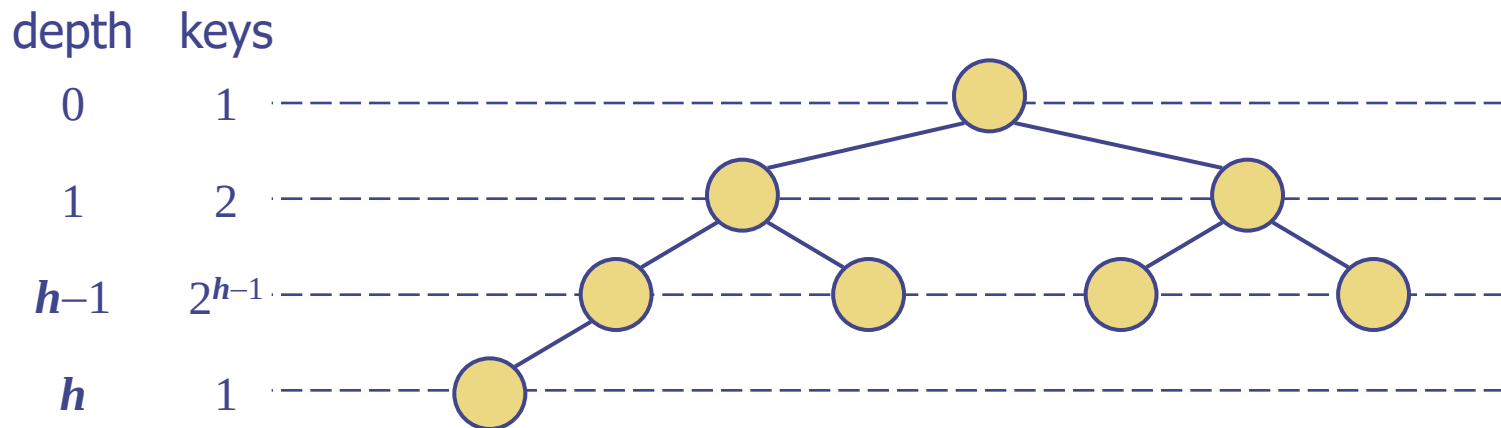
Height of a Heap



- **Theorem:** A heap storing n keys has height $O(\log n)$

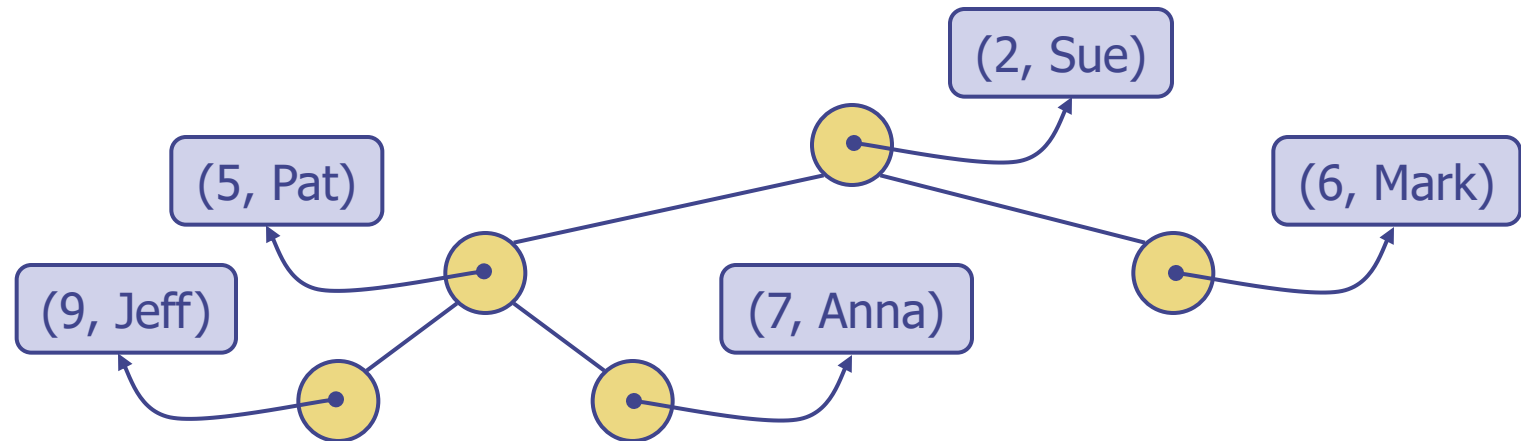
Proof: (we apply the complete binary tree property)

- Let h be the height of a heap storing n keys
- Since there are 2^i keys at depth $i = 0, \dots, h-1$ and at least one key at depth h , we have $n \geq 1 + 2 + 4 + \dots + 2^{h-1} + 1$
- Thus, $n \geq 2^h$, i.e., $h \leq \log n$



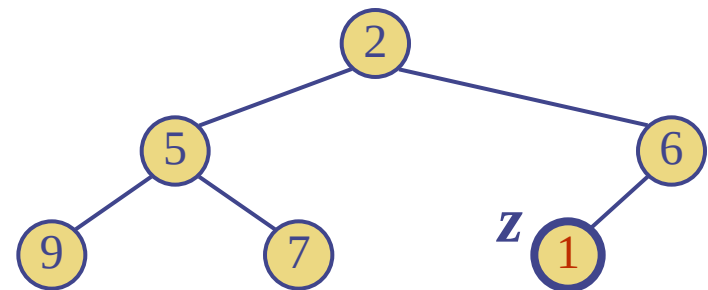
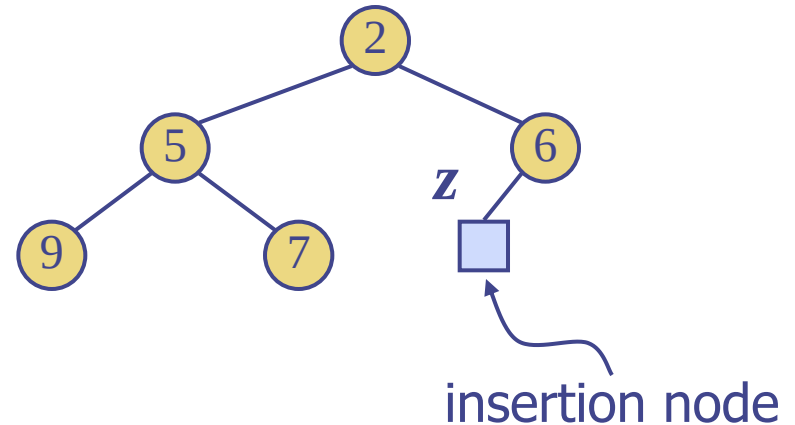
Heaps and Priority Queues

- ❑ We can use a heap to implement a priority queue
- ❑ We store a (key, element) item at each internal node
- ❑ We keep track of the position of the last node



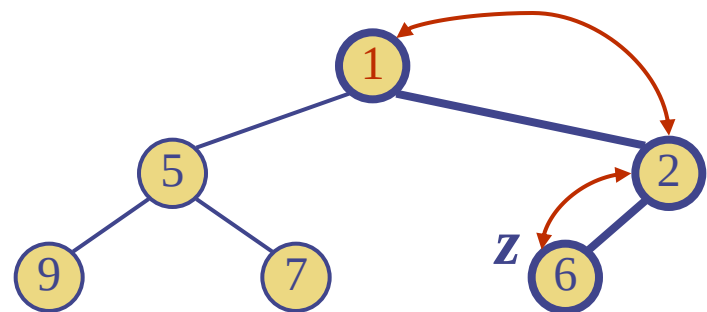
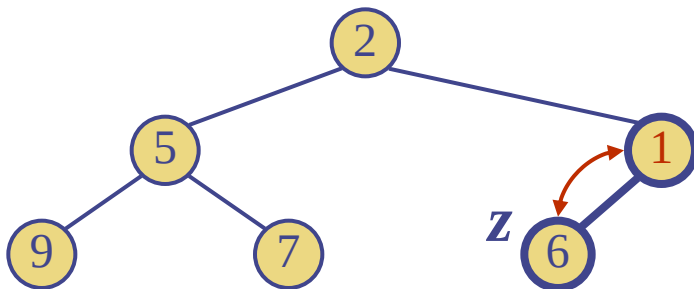
Insertion into a Heap

- ❑ Method add of the priority queue ADT corresponds to the insertion of a key k to the heap
- ❑ The insertion algorithm consists of three steps
 - Find the insertion node z (the new last node)
 - Store k at z
 - Restore the heap-order property (discussed next)



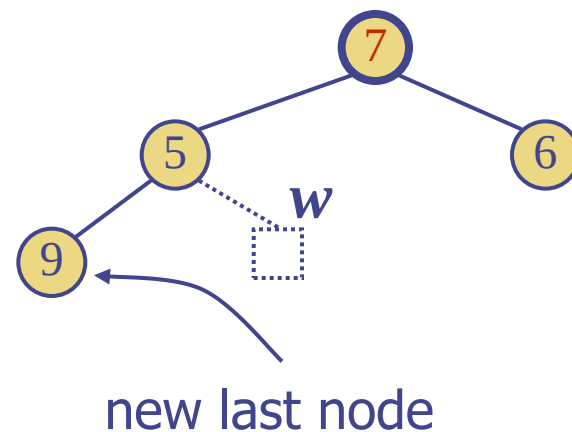
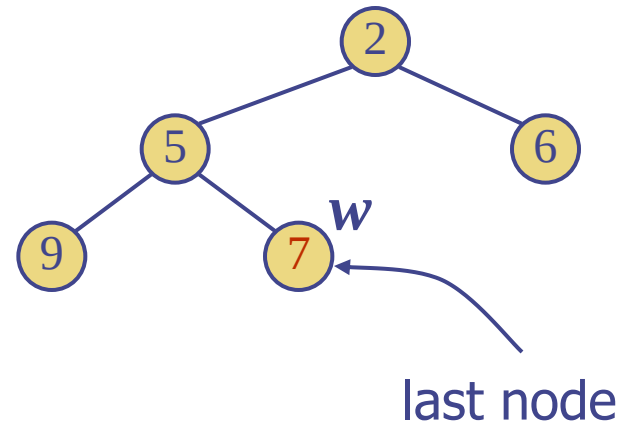
Upheap

- ❑ After the insertion of a new key k , the heap-order property may be violated
- ❑ Algorithm upheap restores the heap-order property by swapping k along an upward path from the insertion node
- ❑ Upheap terminates when the key k reaches the root or a node whose parent has a key smaller than or equal to k
- ❑ Since a heap has height $O(\log n)$, upheap runs in $O(\log n)$ time



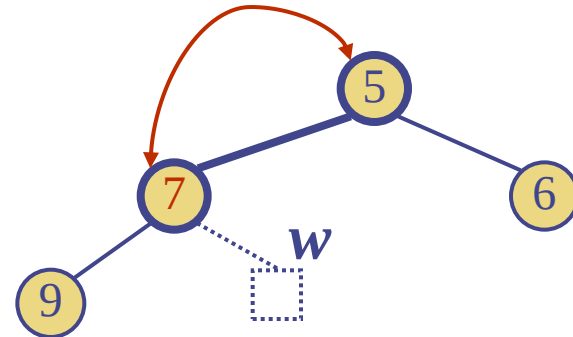
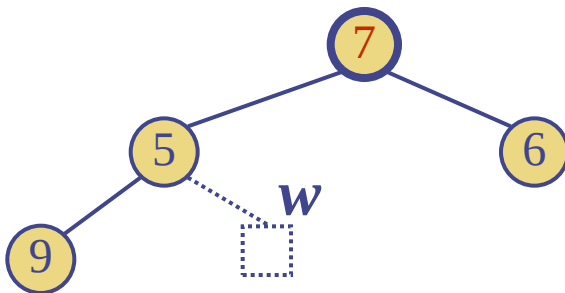
Removal from a Heap

- ❑ Method `remove_min` of the priority queue ADT corresponds to the removal of the root key from the heap
- ❑ The removal algorithm consists of three steps
 - Replace the root key with the key of the last node w
 - Remove w
 - Restore the heap-order property (discussed next)



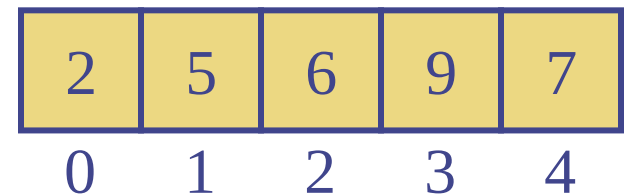
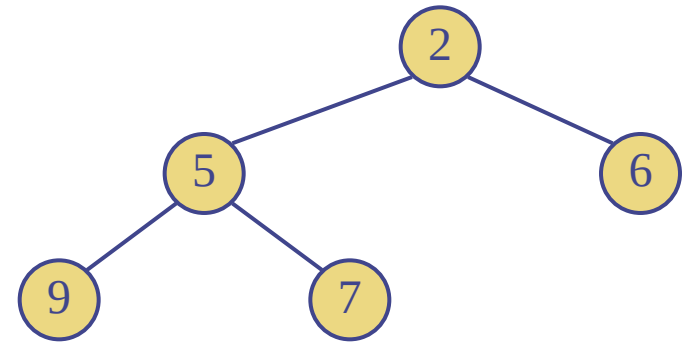
Downheap

- ❑ After replacing the root key with the key k of the last node, the heap-order property may be violated
- ❑ Algorithm downheap restores the heap-order property by swapping key k along a downward path from the root
- ❑ Upheap terminates when key k reaches a leaf or a node whose children have keys greater than or equal to k
- ❑ Since a heap has height $O(\log n)$, downheap runs in $O(\log n)$ time



Array-based Heap Implementation

- We can represent a heap with n keys by means of an array of length n
- For the node at rank i
 - the left child is at rank $2i + 1$
 - the right child is at rank $2i + 2$
- Links between nodes are not explicitly stored
- Operation add corresponds to inserting at rank $n + 1$
- Operation remove_min corresponds to removing at rank 0



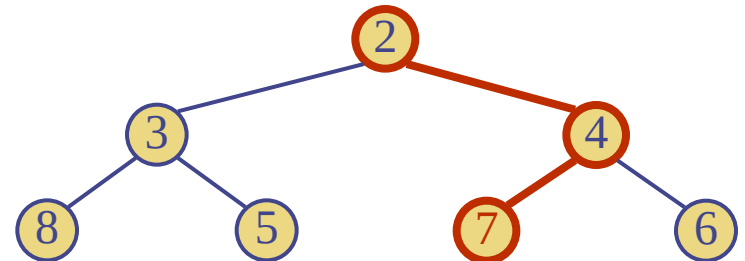
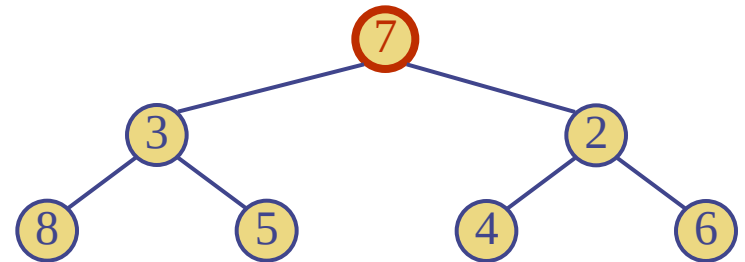
Python Heap Implementation

```
1 class HeapPriorityQueue(PriorityQueueBase): # base class defines _Item
2     """A min-oriented priority queue implemented with a binary heap."""
3     #----- nonpublic behaviors -----
4     def _parent(self, j):
5         return (j-1) // 2
6
7     def _left(self, j):
8         return 2*j + 1
9
10    def _right(self, j):
11        return 2*j + 2
12
13    def _has_left(self, j):
14        return self._left(j) < len(self._data)    # index beyond end of list?
15
16    def _has_right(self, j):
17        return self._right(j) < len(self._data)    # index beyond end of list?
18
19    def _swap(self, i, j):
20        """Swap the elements at indices i and j of array."""
21        self._data[i], self._data[j] = self._data[j], self._data[i]
22
23    def _upheap(self, j):
24        parent = self._parent(j)
25        if j > 0 and self._data[j] < self._data[parent]:
26            self._swap(j, parent)
27            self._upheap(parent)    # recur at position of parent
28
29    def _downheap(self, j):
30        if self._has_left(j):
31            left = self._left(j)
32            small_child = left    # although right may be smaller
33            if self._has_right(j):
34                right = self._right(j)
35                if self._data[right] < self._data[left]:
36                    small_child = right
37            if self._data[small_child] < self._data[j]:
38                self._swap(j, small_child)
39                self._downheap(small_child)    # recur at position of small child
```

```
40    #----- public behaviors -----
41    def __init__(self):
42        """Create a new empty Priority Queue."""
43        self._data = []
44
45    def __len__(self):
46        """Return the number of items in the priority queue."""
47        return len(self._data)
48
49    def add(self, key, value):
50        """Add a key-value pair to the priority queue."""
51        self._data.append(self._Item(key, value))
52        self._upheap(len(self._data) - 1)    # upheap newly added position
53
54    def min(self):
55        """Return but do not remove (k,v) tuple with minimum key.
56
57        Raise Empty exception if empty.
58        """
59        if self.is_empty():
60            raise Empty('Priority queue is empty.')
61        item = self._data[0]
62        return (item._key, item._value)
63
64    def remove_min(self):
65        """Remove and return (k,v) tuple with minimum key.
66
67        Raise Empty exception if empty.
68        """
69        if self.is_empty():
70            raise Empty('Priority queue is empty.')
71        self._swap(0, len(self._data) - 1)    # put minimum item at the end
72        item = self._data.pop()    # and remove it from the list;
73        self._downheap(0)    # then fix new root
74        return (item._key, item._value)
```

Merging Two Heaps

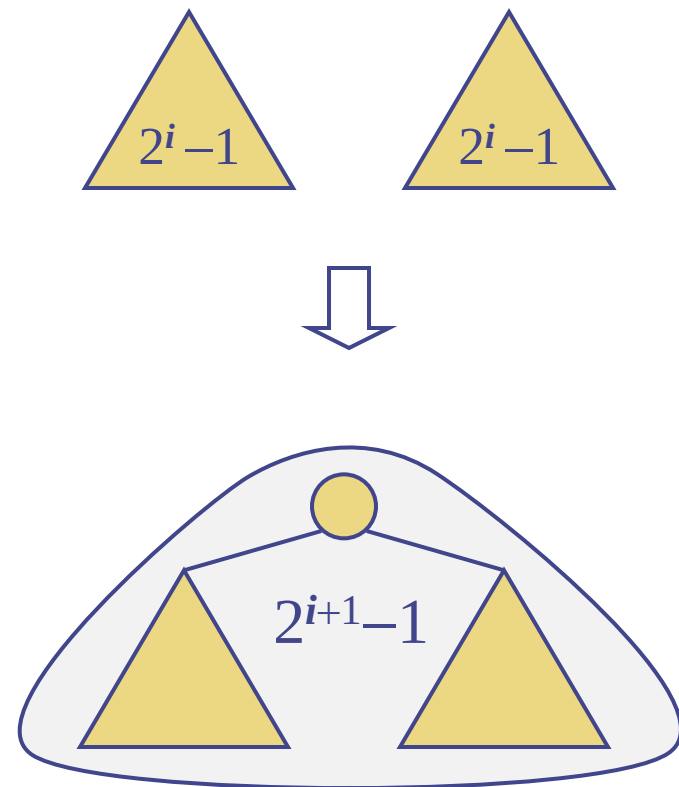
- We are given two two heaps and a key k
- We create a new heap with the root node storing k and with the two heaps as subtrees
- We perform downheap to restore the heap-order property



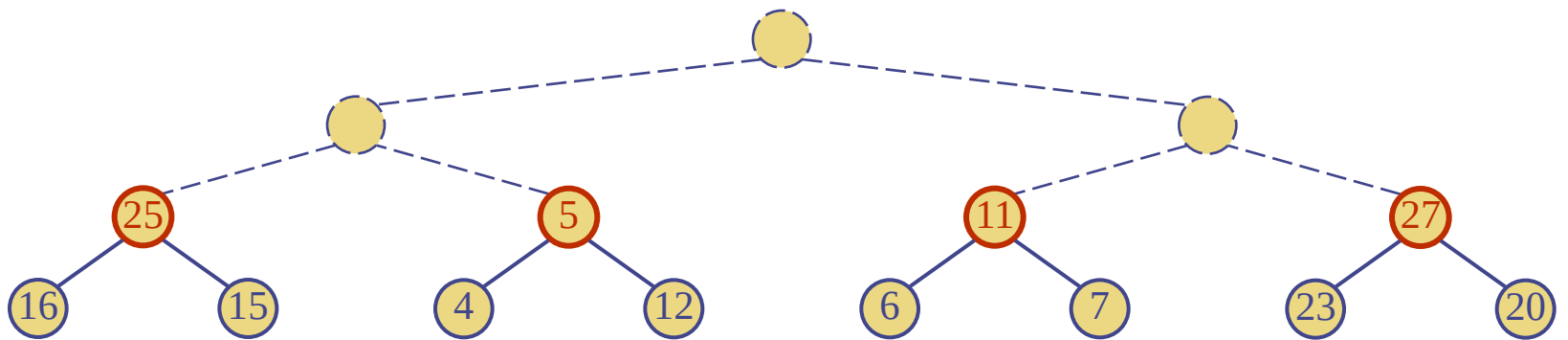
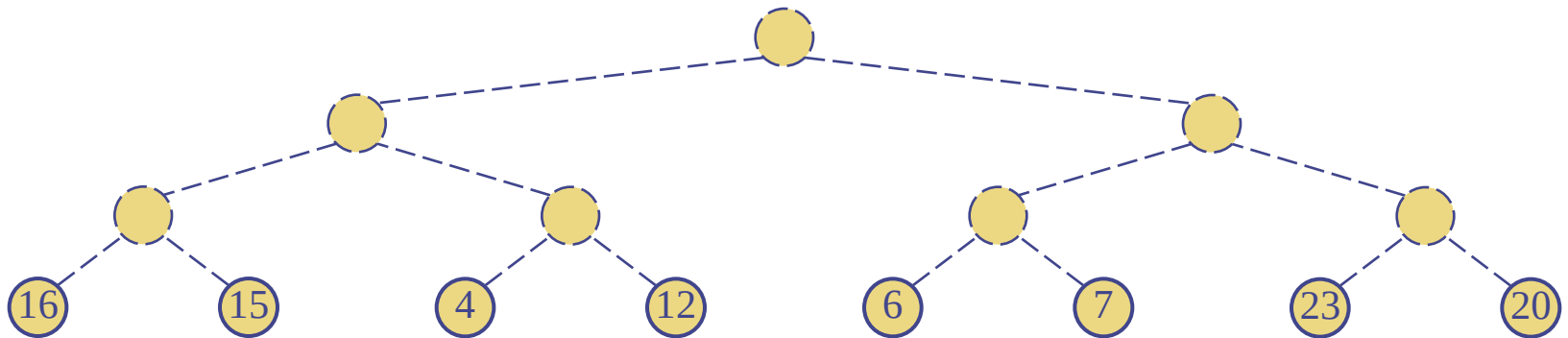
Bottom-up Heap Construction



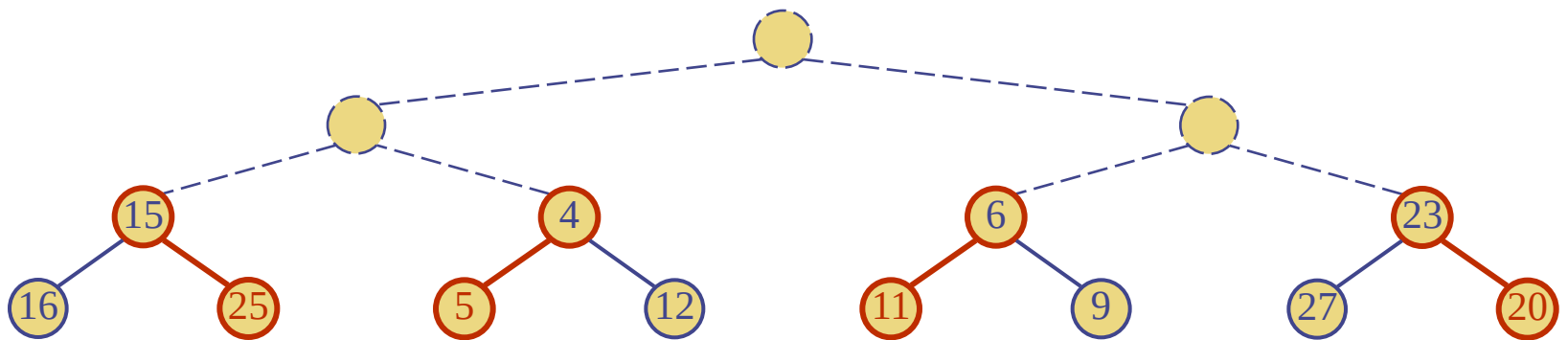
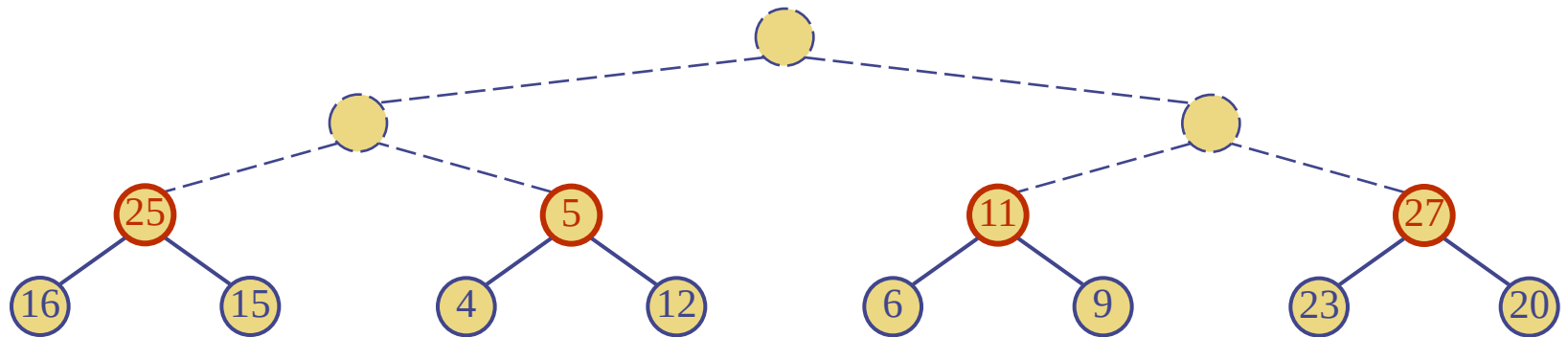
- We can construct a heap storing n given keys in using a bottom-up construction with $\log n$ phases
- In phase i , pairs of heaps with $2^i - 1$ keys are merged into heaps with $2^{i+1} - 1$ keys



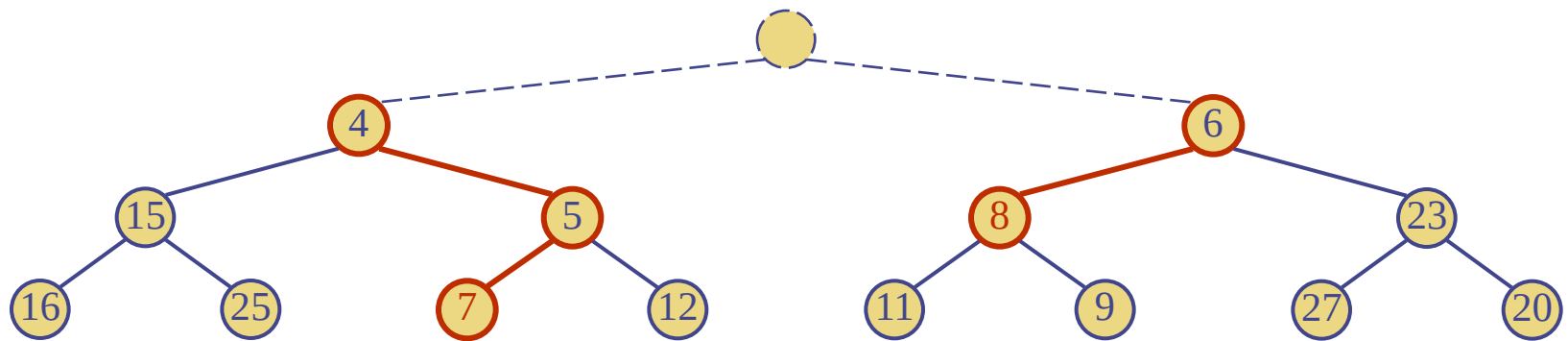
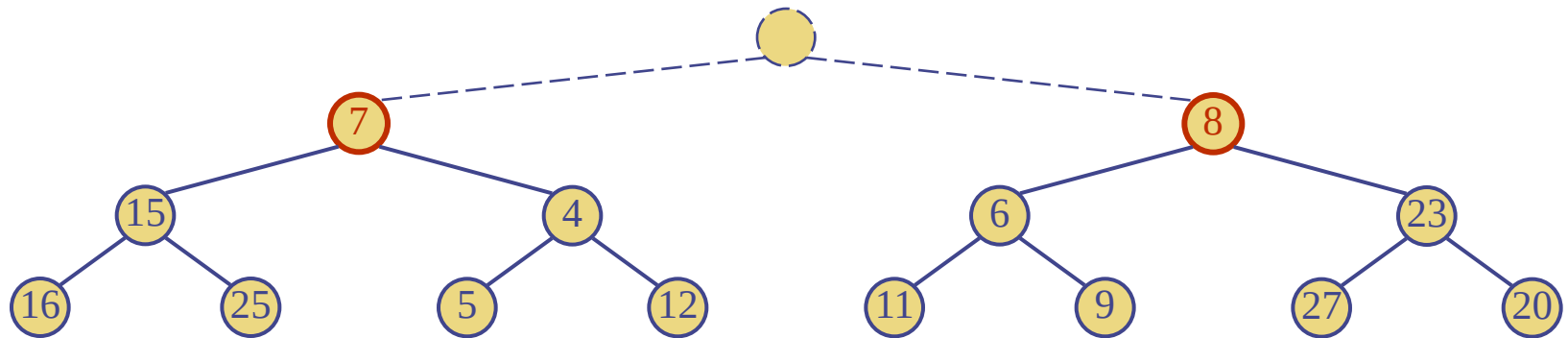
Example



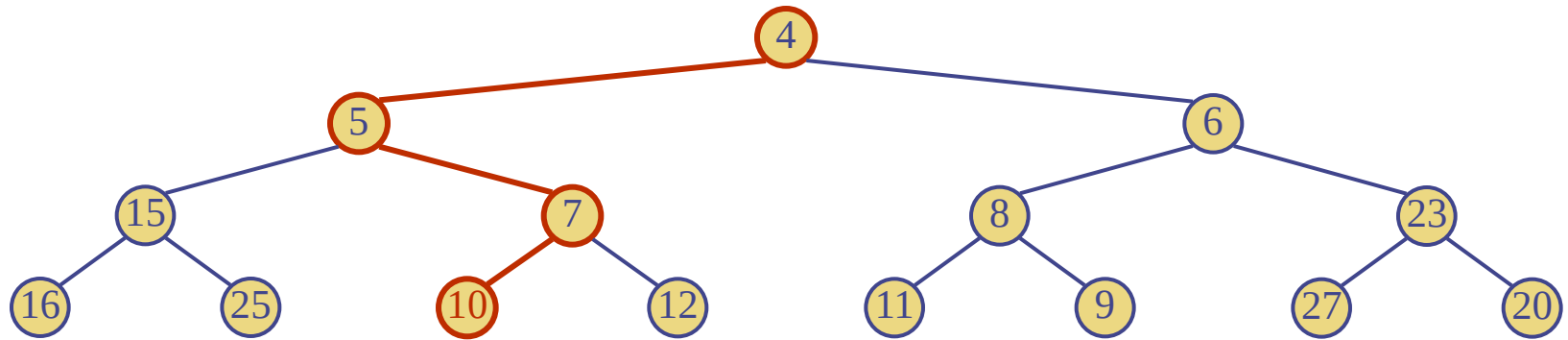
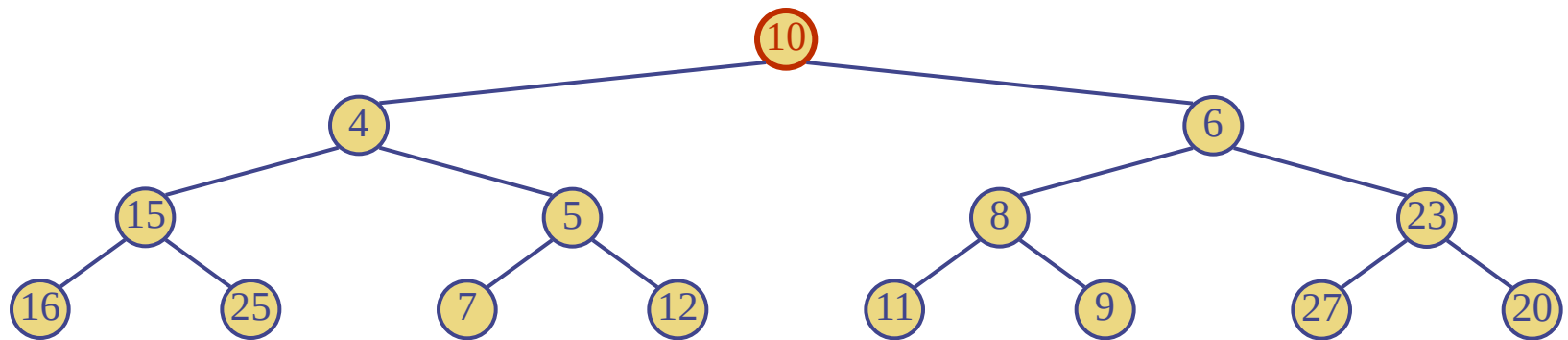
Example (contd.)



Example (contd.)

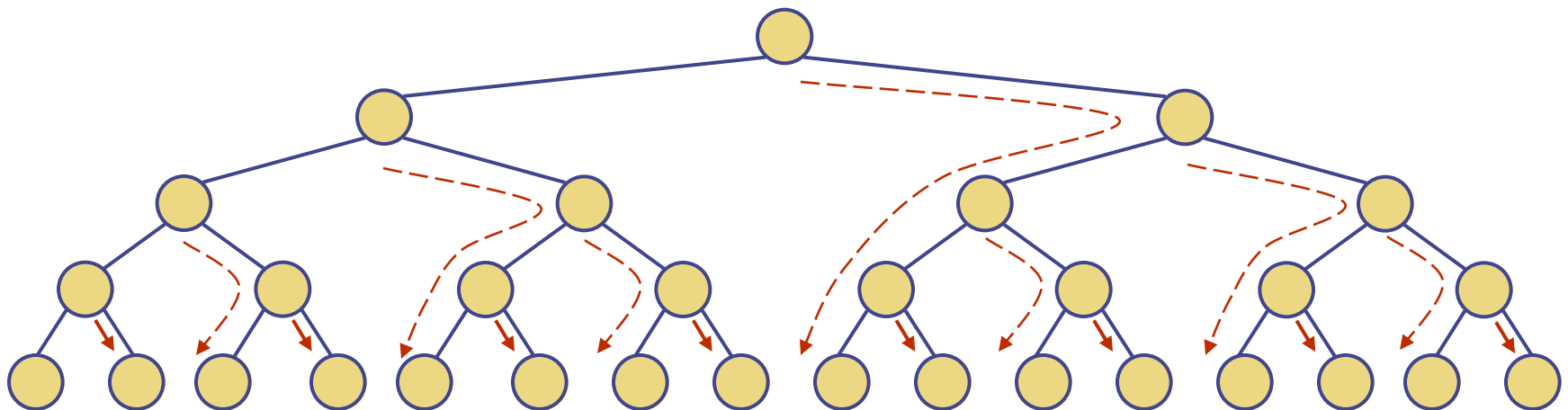


Example (end)

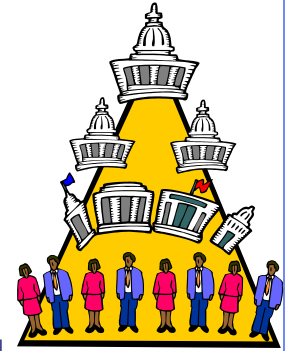


Analysis of Heap Construction

- ❑ We visualize the worst-case time of a downheap with a proxy path that goes first right and then repeatedly goes left until the bottom of the heap (this path may differ from the actual downheap path)
- ❑ Since each node is traversed by at most two proxy paths, the total number of nodes of the proxy paths is $O(n)$
- ❑ Thus, bottom-up heap construction runs in $O(n)$ time
- ❑ Bottom-up heap construction is faster than n successive insertions



Heap-Sort



- Consider a priority queue with n items implemented by means of a heap
 - the space used is $O(n)$
 - methods **add** and **remove_min** take $O(\log n)$ time
 - methods **len**, **is_empty**, and **min** take time $O(1)$ time
- Using a heap-based priority queue, we can sort a sequence of n elements in $O(n \log n)$ time
- The resulting algorithm is called heap-sort
- Heap-sort is much faster than quadratic sorting algorithms, such as insertion-sort and selection-sort

Heap-Sort (in place)

